

Table des matières

Cours 03 — Charger des fichiers dans la couche RAW	1
Objectifs	1
1. Trois façons d'extraire.....	2
2. Chargement complet, chargement incrémental.....	2
3. L'idempotence et le journal des chargements.....	3
4. La couche RAW.....	4
5. Déposer sans transformer : le corpus.....	5
Points clés à retenir.....	6
Exercices.....	6

Cours 03 — Charger des fichiers dans la couche RAW

Théorie · Module 3/12 — Formation Data Engineering · ~20 min · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution. Le Cours 03 fait entrer dans l'entrepôt les fichiers que livrent les systèmes de Boréal, sans les perdre ni les doubler.

En 30 secondes :

- Une donnée s'extrait de trois façons : d'un dépôt de fichiers, d'une base par un pilote comme ODBC, d'une API REST. Ce cours traite la première, la plus répandue ; le Cours 04, les deux autres.
- Un chargement doit pouvoir se relancer sans rien doubler : c'est l'idempotence, et un journal des chargements la rend possible.
- La couche RAW garde ce qui est livré, tel quel, avec la trace de son origine. Elle ne corrige rien, pas même un extrait livré deux fois.
- Toute source ne devient pas une table : la documentation se dépose telle quelle dans le bronze, sans rien doubler elle non plus.

L'idée du cours. Au Cours 02, le bronze rangeait deux sources en Parquet. Ici, les fichiers que livrent les systèmes de Boréal entrent dans l'entrepôt, 22 tables en tout, par un chargeur qu'on relance sans crainte. Tout se fait sous Windows, avec Python et DuckDB : aucun service à allumer.

Objectifs

- Distinguer les trois voies d'extraction (fichiers, base, API) et ce que chacune garantit.

- Choisir entre chargement complet et chargement incrémental selon la forme d'une livraison.
- Rendre un chargement idempotent par un journal, et reconnaître ce qu'un journal laisse passer.
- Décrire la forme de la couche RAW et justifier chacune de ses règles.
- Déposer tel quel ce qui ne devient pas une table, et rendre ce dépôt idempotent lui aussi.

1. Trois façons d'extraire

Labo : Labo_03.md, étape 2.

Une source ne livre pas toujours de la même façon. Boréal en montre trois, et chacune change ce que l'ingénieur de données doit faire.

Voie	Chez Boréal	Qui agit	Ce que la source garantit	Ce qui reste à votre charge
Un dépôt de fichiers	les extraits de l'ERP et du portail des garanties, l'archive de la télémétrie, les exports de l'usine	la source écrit, vous ramassez	le fichier, tel qu'écrit	savoir quels fichiers sont nouveaux, et n'en charger aucun deux fois
Une base, par un pilote	la base de l'ERP, dans PostgreSQL	vous interrogez	une réponse cohérente au moment de la lecture	ne pas surcharger la base, et ne lire que ce qui a changé
Une API REST	les observations d'ECCC	vous demandez, page par page	chaque réponse, avec son code HTTP	parcourir toutes les pages, réessayer après une panne, compter ce qui est reçu

Le dépôt de fichiers est la voie la plus répandue en entreprise, et la plus simple en apparence. Sa difficulté se cache dans le temps : un fichier arrive en retard, arrive deux fois, ou revient corrigé sous le même nom. La base et l'API, elles, se lisent à la demande ; leur difficulté est de ne rien manquer sans tout relire.

Ce cours traite le dépôt de fichiers, la voie la plus répandue. Le Cours 04 lit la base et l'API.

2. Chargement complet, chargement incrémental

Labo : Labo_03.md, étape 2.

Deux formes de livraison se croisent chez Boréal.

Forme	Ce que contient chaque livraison	Chez Boréal	Ce qu'on en fait
Complète	toute la table, dans son état du jour	le registre des actifs, la GMAO, l'instantané météo, le catalogue du corpus ; et, chaque mois, les données maîtres de l'ERP (T005U, KNA1, MARA, MAK1)	la nouvelle version remplace l'ancienne, ou s'ajoute à côté si l'on veut garder l'historique
Incrémentale	ce qui a été créé ou modifié depuis la livraison précédente	les commandes, leurs postes et les numéros de série de l'ERP ; les extraits des garanties ; les jours de télémétrie	chaque livraison s'ajoute aux précédentes

Un chargement complet est simple et sûr : on remplace, et l'état est juste. Il coûte cher dès que la table grossit, et il efface la version de la veille. Un chargement incrémental coûte peu et garde tout, mais il exige de savoir ce qui est déjà chargé, sans quoi il double.

Une conséquence surprend souvent : un extrait incrémental n'est pas une copie de la base. Au Cours 01, les extraits de l'ERP comptaient 26 538 lignes d'en-têtes pour 25 483 commandes, parce qu'une commande modifiée le mois suivant revient dans l'extrait suivant, dans sa nouvelle version. Pour retrouver l'état de la base, on garde, pour chaque commande, la version du lot le plus récent. L'étape 2 du Labo 04 le fait, pour prouver que rien ne manque, et retrouve la base à la ligne près.

Les données maîtres de l'ERP montrent l'autre face. Livrées en entier chaque mois, elles entrent quand même lot après lot dans la RAW, qui garde ainsi les 49 versions du fichier des concessionnaires : 7 481 lignes pour 156 concessionnaires, de 150 à 156 selon le mois. C'est la mémoire de chaque ouverture et de chaque fermeture, que le remplacement aurait effacée.

3. L'idempotence et le journal des chargements

Labo : Labo_03.md, étapes 1 et 2.

Un traitement est idempotent quand le relancer donne le même résultat que le lancer une fois. Le Cours 01 en faisait la propriété qui gouverne la formation. Elle compte ici plus qu'ailleurs, parce qu'un chargement se relance tout le temps : après une panne au milieu du travail, quand l'orchestrateur rattrape une nuit manquée (Cours 06), quand un collègue veut simplement vérifier.

Un INSERT n'est pas idempotent. Chargez deux fois l'extrait de décembre 2025 dans une table : 854 lignes pour 427 commandes, et aucune erreur. Trois stratégies protègent un chargement :

Stratégie	Comment	Quand elle convient
Remplacer	vider la cible, puis tout recharger	les livraisons complètes et les petites tables

Stratégie	Comment	Quand elle convient
Fusionner	insérer ou mettre à jour selon une clé (MERGE, INSERT ... ON CONFLICT)	une source qui a une clé fiable, quand on ne veut que l'état courant
Tenir un journal	noter chaque fichier chargé, et ne charger que ceux qu'on ne connaît pas	les livraisons de fichiers, quand on veut garder chaque version

La RAW de Boréal garde chaque version : elle tient un journal. Le chargeur du labo note, pour chaque fichier, son chemin, son empreinte, son lot, ses lignes et l'heure du chargement, dans la table `pipeline.chargements`. L'empreinte est un résumé calculé sur le contenu, un SHA-256 : deux fichiers identiques ont la même, un seul octet changé en donne une autre. Au second passage, le chargeur compare chaque fichier au journal et ne charge rien : 0 fichier, en moins d'une seconde.

Le fichier	Ce que fait le chargeur
inconnu du journal	il le charge et le note
connu, avec la même empreinte	il le saute
connu, avec une autre empreinte	il retire les lignes de l'ancienne version, charge la nouvelle et la note : c'est ainsi qu'un export complet se met à jour

Le chargement d'un fichier et sa ligne au journal s'écrivent dans la même transaction. Si le chargeur s'arrête au milieu, ni la table ni le journal ne gardent une moitié de livraison, et le passage suivant reprend proprement.

Piège — ce que le journal laisse passer. Le journal reconnaît un fichier par son chemin. L'extrait de mars 2023 a été livré deux fois, sous deux noms : 2023-03 et 2023-03_bis. Pour le journal, ce sont deux fichiers, et 609 commandes entrent deux fois, sans erreur. Reconnaître un fichier par sa seule empreinte attraperait ce doublon, mais au prix d'une autre erreur : les 49 fichiers T005U des extraits ont exactement le même contenu, et seul le premier serait chargé. La RAW n'aurait plus de régions après le lot 2022-01, et qui prend le lot le plus récent n'en trouverait aucune. Le doublon reste donc dans la RAW, marqué par son lot, et c'est un contrat de données qui le signale (Cours 11).

Pourquoi — « au moins une fois », puis idempotent. Aucun transport ne garantit à lui seul qu'un fichier ou un message arrive exactement une fois : ou il risque de le perdre, ou il accepte de le doubler. L'ingénierie des données choisit presque toujours de doubler plutôt que de perdre, puis rend chaque étape idempotente. Livraison au moins une fois, traitement idempotent : l'effet est celui d'une livraison unique.

4. La couche RAW

Labo : Labo_03.md, étapes 1, 2 et 3.

La RAW est le produit que cette formation livre à l'ingénieur analytique : les fichiers livrés, devenus des tables de l'entrepôt, sans rien perdre ni rien décider à la place de ceux qui connaissent les règles métier. Chacune de ses règles répond à un piège déjà vu.

Règle	Pourquoi
une table par fichier livré : raw.erp_vbak, raw.garanties_clients, raw.meteo_quotidien	le consommateur retrouve la source à partir du nom
toutes les colonnes en texte, avec leurs noms tels que livrés	les zéros de tête (0000200154) et les dates vides 00000000 survivent ; aucun type n'est deviné (Cours 01)
un objet JSON dans une colonne JSON	la réclamation et le message gardent leur forme, et se lisent par chemin (message ->> 'sernr')
un champ vide devient NULL	la convention par défaut de Snowflake et de Databricks : la RAW se charge pareil partout
trois colonnes de lignage : _fichier, _lot, _charge_le	chaque ligne dit d'où elle vient et quand elle est arrivée ; on filtre un lot, on rejoue un fichier, on retrouve une anomalie à sa source
ce qui est livré en double reste, marqué par son lot	le brut n'est plus le brut dès qu'on le corrige, et le test d'unicité de l'ingénieur analytique n'aurait plus rien à prouver

La RAW ne décode aucun code, ne dédouble rien et ne joint aucune source. Elle garde les 609 commandes de 2023-03_bis, les 2 303 doublons de la télémétrie (158 146 messages pour 155 843 identifiants) et chaque version d'une réclamation (5 623 objets pour 3 348 réclamations). Tirer l'état courant d'une table est le premier geste du STAGING d'Analytics Engineering.

Le bronze du Cours 02 et la RAW se ressemblent, mais ne servent pas les mêmes lecteurs. Le bronze range des fichiers dans le lac, en Parquet, pour qui lit des fichiers ; la RAW range des tables dans l'entrepôt, pour qui écrit du SQL et du dbt. La télémétrie vit dans les deux. Les points bruts de l'historien et du compteur, eux, restent au bronze : leurs séries nettoyées arriveront dans le schéma argent au Cours 08.

À retenir — la RAW garde aussi ce qu'on a demandé d'effacer. Le client CL0011163 a demandé l'effacement de ses données le 2025-09-16 : à partir du lot 2025-09, ses nom, courriel et adresse arrivent vides, mais le lot 2024-01 les contient encore. Une couche qui garde tout garde aussi les données personnelles des livraisons passées. Ce que la Loi 25 exige alors (purger, chiffrer, restreindre l'accès) se décide avec la gouvernance : c'est la matière de la formation Architecture et Gouvernance.

5. Déposer sans transformer : le corpus

Labo : Labo_03.md, étape 4.

Toute source ne devient pas une table. La documentation de Boréal (manuels, bulletins de service, politiques de garantie, procédures, guides) se dépose telle quelle dans le bronze, avec son arborescence : 35 fichiers, dont 33 documents, le catalogue index.csv et la note de l'équipe. Seul le catalogue entre aussi dans la RAW, raw.corpus_index, parce qu'il se lit en SQL.

Découper les documents en passages et les indexer pour une recherche appartient à la formation AI Engineering. Le dépôt, lui, doit être idempotent comme le reste : le script du labo compare le contenu de chaque fichier à celui déjà déposé, et ne recopie que ce qui a changé.

Snowflake. `COPY INTO` tient son propre journal : pendant 64 jours, la table garde le nom, la taille et l’empreinte de chaque fichier chargé, et refuse de le charger à nouveau. L’option `FORCE` passe outre, et double les lignes. Snowpipe, le chargement en continu, garde cet historique 14 jours. Le nom du fichier se lit par la colonne `METADATA$FILENAME`. L’annexe Snowflake du Labo 03 refait le chargement de VBAK dans un schéma transient.

Databricks. `COPY INTO` saute les fichiers déjà chargés, même s’ils ont été modifiés depuis : contrairement au chargeur du labo, un fichier corrigé sous le même nom n’est pas rechargé, sauf avec l’option `force`. Auto Loader fait le même travail en continu, en gardant la trace des fichiers traités dans son point de reprise. Le chemin du fichier se lit par la colonne cachée `_metadata`. L’annexe Databricks du Labo 03 charge VBAK depuis un volume Unity Catalog.

Recoupements — ce que ce cours partage avec ses voisines. - Analytics Engineering. Les codes SAP que la RAW garde tels quels (KUKLA, ABGRU, AUART) se décodent au STAGING, et l’état courant d’une table s’y tire des lots. DE livre la RAW ; AE commence au STAGING. - Snowflake. `COPY INTO` et Snowpipe se voient côté plateforme dans la formation Snowflake ; ici, la mécanique générale, celle qu’on retrouve dans tout outil de chargement.

Points clés à retenir

- Trois voies d’extraction : le dépôt de fichiers, qu’on ramasse ; la base, qu’on interroge par un pilote ; l’API, qu’on tire page par page. Chacune laisse une responsabilité différente au client.
 - Un chargement complet remplace, un chargement incrémental ajoute. Un extrait incrémental n’est pas la base : l’état se retrouve en gardant la version la plus récente de chaque clé.
 - Un chargement se relance toujours un jour : il doit être idempotent. Un journal des chargements, tenu dans la même transaction que les données, le garantit pour des fichiers.
 - Le journal laisse passer un fichier livré deux fois sous deux noms, et c’est voulu : la RAW le garde marqué par son lot, et un contrat le signale.
 - La RAW garde tout, en texte, avec son lignage. Elle ne décode, ne dédoublonne et ne joint rien.
 - Ce qui ne se lit pas en SQL se dépose tel quel, et le dépôt est idempotent comme le reste : il compare les contenus, pas les dates.
-

Exercices

Les exercices de ce cours vivent dans `Exercices_03.md`, et leur corrigé dans `Corrige_03.md`. Faites-les après le labo : plusieurs énoncés supposent que la RAW est chargée.